

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.24

DATABASE INTERCEPTORS

 Copy as Markdown

Database interceptors

A database interceptor lets you wrap every database operation of a session in the same behavior, such as query tracing, tenant scoping, or a guard that rejects writes in a protected environment. Serverpod calls the interceptor once when it creates a session, passing the session and the framework database, and whatever the interceptor returns becomes `session.db` for the rest of that session.

Register the interceptor when you construct `Serverpod`:

```
var pod = Serverpod(  
  args,  
  databaseInterceptor: (session, inner) {  
    session.log('Created a database layer for this session.');
```

Returning `inner` leaves the framework database in place, so this example only observes session creation. Repeated reads of `session.db` return the same instance for the lifetime of the session.

Return a custom database layer

To intercept the operations themselves, return your own implementation of the `Database` interface exported by `package:serverpod/serverpod.dart`:

```
var pod = Serverpod(  
  args,  
  databaseInterceptor: (session, inner) {  
    return PolicyDatabase(  
      session: session,  
      inner: inner,  
    );  
  },  
);
```



```
);
```

The `Database` class has no public constructor to subclass, so implement the interface and let your IDE generate the overrides. Hold on to the supplied `inner` database and delegate every operation the layer does not change. The object you return becomes `session.db`, so generated calls such as `User.db.find(session)` pass through it as well.

Typical uses are:

- Recording query traces, timings, and metrics.
- Applying a tenant scope to every supported query.
- Enforcing read and write policies.
- Blocking unsafe operations or writes in protected environments.

Pass transaction objects, [runtime parameters](#), ordering, pagination, [row locks](#), and return options through without changing their meaning. A layer that enforces a policy also needs to cover [raw access](#) and transactions, otherwise a caller can reach the database around it. Do not hold on to the session or its inner database after the session closes.

WARNING

Custom database layers build on the `Database` API from `serverpod_database`, which can change in a breaking way in a minor Serverpod release. Review and test your implementation whenever you upgrade Serverpod.